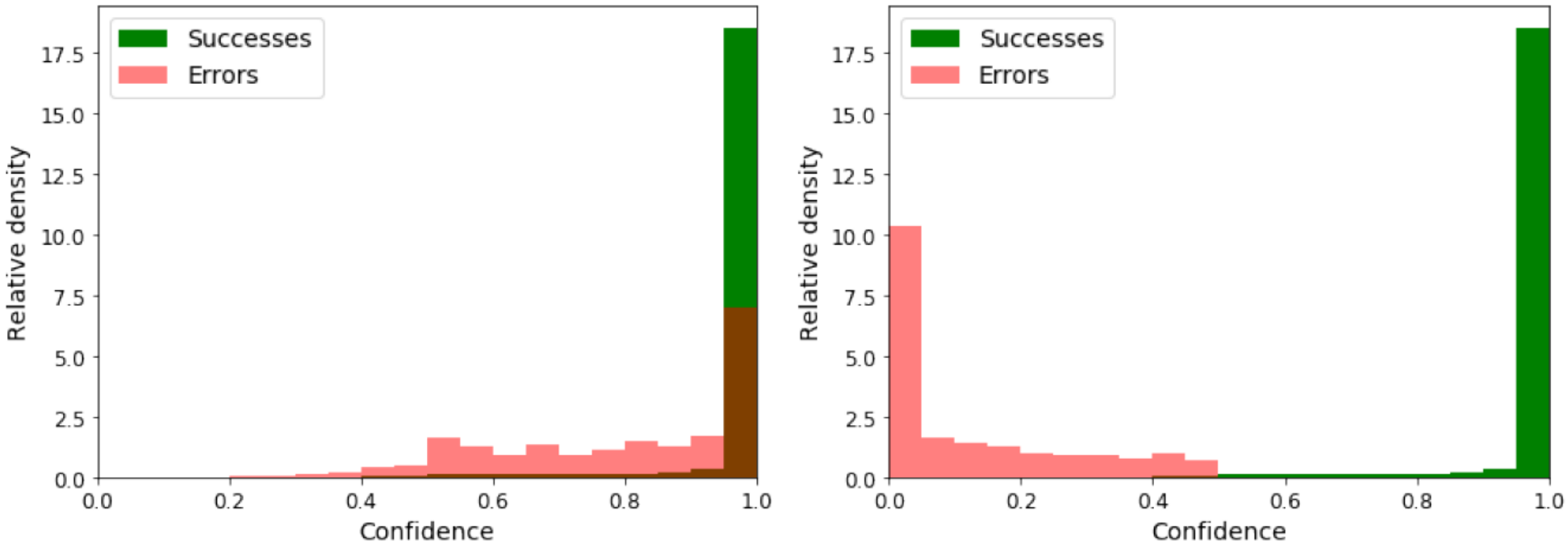
C. Corbière, N. Thome, A. Bar-Hen, M. Cord, and P. Pérez, "Addressing Failure Prediction by Learning Model Confidence," in NeurIPS, 2019.

2020.5.8
胡敬玉

问题

Maximum Class Probability

True Class Probabiliy



(a) Maximum Class Probability

(b) Our Proposal (True Class Probability)

定义

Let us consider a dataset $\mathcal{D}$ which consists of N *i.i.d.* training samples $\mathcal{D} = \{(\mathbf{x}_i, y_i^*)\}_{i=1}^N$ where $\mathbf{x}_i \in \mathbb{R}^d$ is a d-dimensional feature and $y_i^* \in \mathcal{Y} = \{1, ..., K\}$ is its true class. We view a classification neural network as a probabilistic model: given an input $\mathbf{x}$, the network assigns a probabilistic predictive distribution $P(Y|\mathbf{w}, \mathbf{x})$ by computing the softmax output for each class k and where $\mathbf{w}$ are the parameters of the network. From this predictive distribution, one can infer the class predicted by the model as $\hat{y} = \underset{k \in \mathcal{Y}}{\operatorname{argmax}}\, P(Y = k|\mathbf{w}, \mathbf{x})$.

During training, network parameters $\mathbf{w}$ are learned following a maximum likelihood estimation framework where one minimizes the Kullback-Leibler (KL) divergence between the predictive distribution and the true distribution. In classification, this is equivalent to minimizing the cross-entropy loss w.r.t. $\mathbf{w}$, which is the negative sum of the log-probabilities over positive labels:

$$\mathcal{L}_{\mathrm{CE}}(\mathbf{w}; \mathcal{D}) = -\frac{1}{N} \sum_{i=1}^{N} y_i^* \log P(Y = y_i^*|\mathbf{w}, \mathbf{x}_i). \tag{1}$$

定义

For a given input $\mathbf{x}$, a standard approach is to compute the softmax probability of the predicted class $\hat{y}$, that is the Maximum Class Probability: $\mathrm{MCP}(\mathbf{x}) = \max_{k \in \mathcal{Y}} P(Y = k|\mathbf{w}, \mathbf{x}) = P(Y = \hat{y}|\mathbf{w}, \mathbf{x})$.

By taking the largest softmax probability, MCP leads to high confidence values both for errors and correct predictions, making it hard to distinguish them, as shown in Figure 1(a). On the other hand, when the model is misclassifying an example, the probability associated to the true class y^* would be more likely close to a low value, reflecting the fact that the model made an error. Thus, we propose to consider the True Class Probability as a suitable confidence criterion for failure prediction:

$$\begin{aligned} \mathrm{TCP}: \quad & \mathbb{R}^d \times \mathcal{Y} \rightarrow \mathbb{R} \\ & (\mathbf{x}\ ,\ y^*) \rightarrow P(Y = y^*|\mathbf{w}, \mathbf{x}) \end{aligned} \tag{2}$$

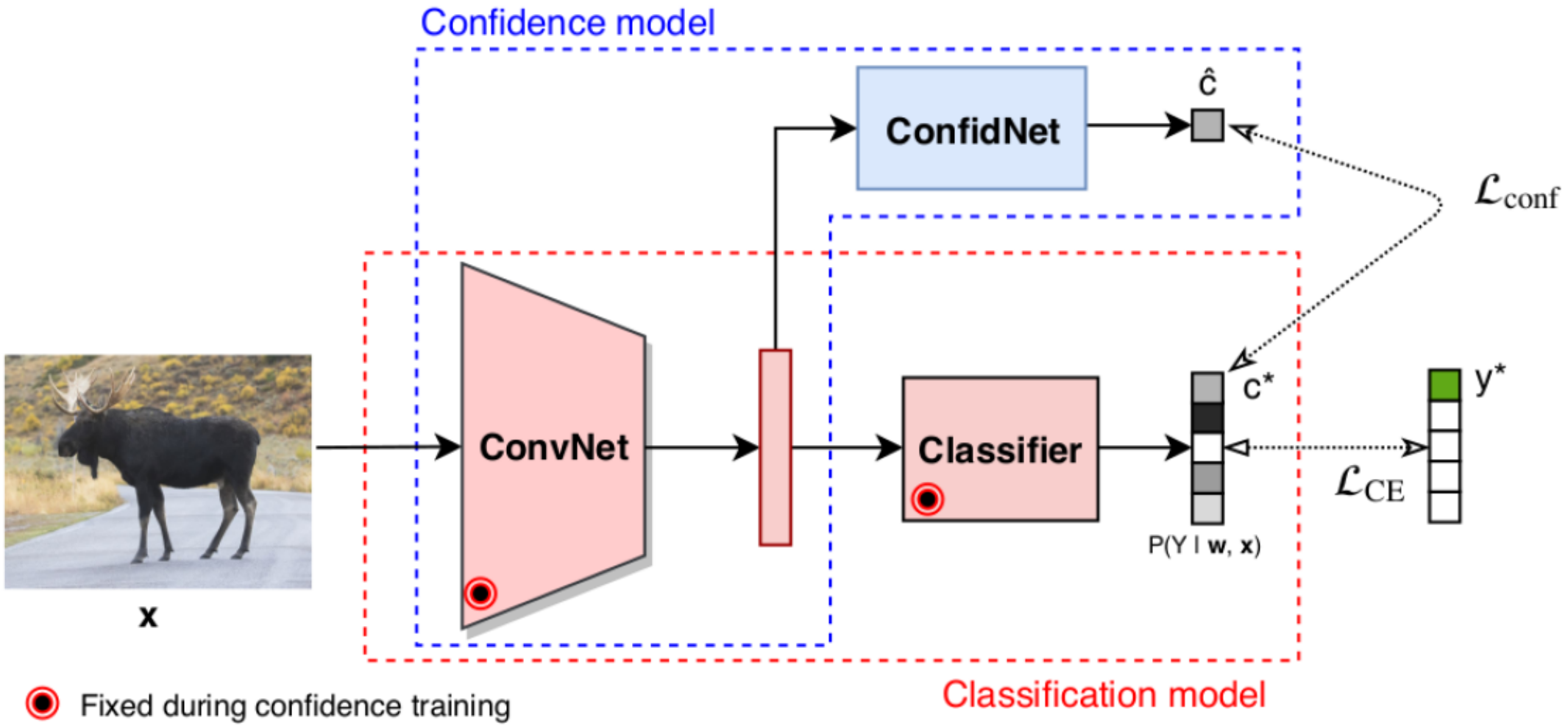


Figure 2: Our approach is based on two sub-networks. The classification model with parameters $\mathbf{w}$ is composed of a succession of convolutional and dense layers ('ConvNet') followed by a final dense layer with softmax activation. The confidence network, 'ConfidNet', builds upon features maps extracted by ConvNet, and is composed of a succession of layers which output a confidence score $\hat{c}(\mathbf{x}, \theta) \in [0, 1]$.

Learning scheme. Our complete confidence model, from input image to confidence score, shares its first encoding part ('ConvNet' in Fig.2) with the classification model M. The training of ConfidNet

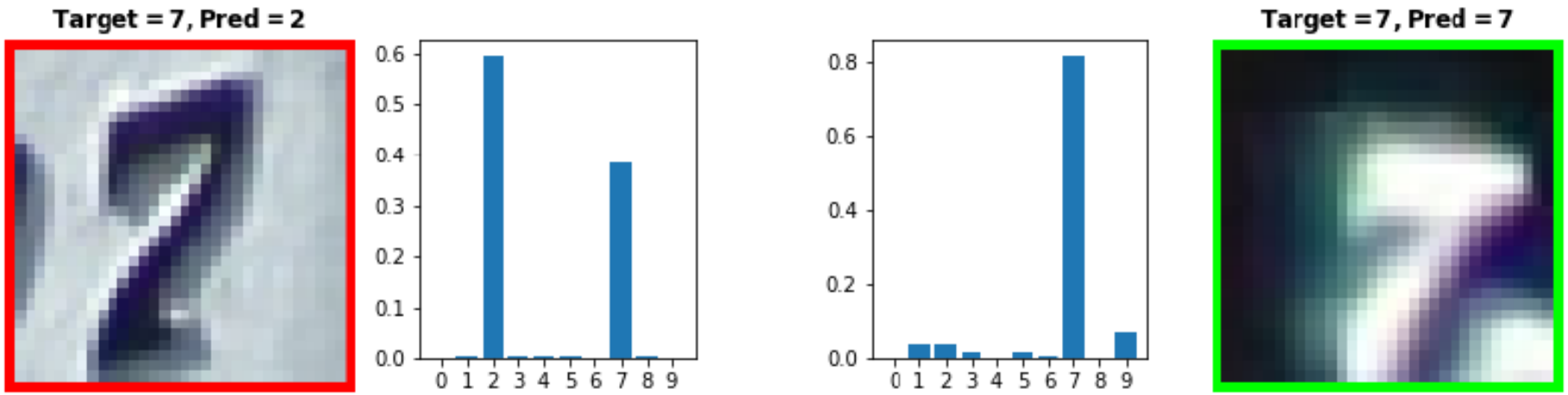
[1] or its normalized variant $\mathrm{TCP}^r(\mathbf{x}, y^*)$.

starts by fixing entirely M (freezing $\mathbf{w}$) and learning θ using loss (4). In a next step, we can then fine-tune the ConvNet encoder. However, as model M has to remain fixed to compute similar classification predictions, we have now to decouple the feature encoders used for classification and confidence prediction respectively. We also deactivate dropout layers in this last training phase and reduce learning rate to mitigate stochastic effects that may lead the new encoder to deviate too much from the original one used for classification. Data augmentation can thus still be used.

$$\mathcal{L}_{\mathrm{conf}}(\theta; \mathcal{D}) = \frac{1}{N} \sum_{i=1}^{N} (\hat{c}(\mathbf{x}_i, \theta) - c^*(\mathbf{x}_i, y_i^*))^2.$$

Table 1: Comparison of failure prediction methods on various datasets. All methods share the same classification network. Note that for MCDropout, test accuracy is averaged over random sampling. All values are percentages.

Dataset	Model	FPR-95%-TPR	AUPR-Error	AUPR-Success	AUC
MNIST MLP	Baseline (MCP) [17]	14.87	37.70	99.94	97.13
	MCDropout [10]	15.15	38.22	99.94	97.15
	TrustScore [20]	12.31	52.18	99.95	97.52
	ConfidNet (Ours)	**11.79**	**57.37**	**99.95**	**97.83**
MNIST Small ConvNet	Baseline (MCP) [17]	5.56	35.05	99.99	98.63
	MCDropout [10]	5.26	38.50	99.99	98.65
	TrustScore [20]	10.00	35.88	99.98	98.20
	ConfidNet (Ours)	**3.33**	**45.89**	**99.99**	**98.82**
SVHN Small ConvNet	Baseline (MCP) [17]	31.28	48.18	99.54	93.20
	MCDropout [10]	36.60	43.87	99.52	92.85
	TrustScore [20]	34.74	43.32	99.48	92.16
	ConfidNet (Ours)	**28.58**	**50.72**	**99.55**	**93.44**
CIFAR-10 VGG16	Baseline (MCP) [17]	47.50	45.36	99.19	91.53
	MCDropout [10]	49.02	46.40	**99.27**	92.08
	TrustScore [20]	55.70	38.10	98.76	88.47
	ConfidNet (Ours)	**44.94**	**49.94**	99.24	**92.12**
CIFAR-100 VGG16	Baseline (MCP) [17]	67.86	71.99	92.49	85.67
	MCDropout [10]	64.68	72.59	**92.96**	86.09
	TrustScore [20]	71.74	66.82	91.58	84.17
	ConfidNet (Ours)	**62.96**	**73.68**	92.68	**86.28**
CamVid SegNet	Baseline (MCP) [17]	63.87	48.53	96.37	84.42
	MCDropout [10]	62.95	49.35	96.40	84.58
	TrustScore [20]		20.42	92.72	68.33
	ConfidNet (Ours)	**61.52**	**50.51**	**96.58**	**85.02**



(a) MCP=0.596, MCDropout=-0.787, *ConfidNet*=0.449 (b) MCP=0.816, MCDropout=-0.786, *ConfidNet*=0.894

Figure 4: Illustrating the limits of MCDropout with entropy as confidence estimation on SVHN test samples. Red-border image (a) is misclassified by the classification model; green-border image (b) is correctly classified. Prediction exhibit similar high entropy in both cases. For each sample, we provide a plot of their softmax predictive distribution.

Table 4: Effect of loss and normalized criterion on AUPR-Error

Dataset		Loss	Criterion
	TCP	BCE	TCP^r
CIFAR-10	49.94%	47.95%	48.78%
CamVid	50.51%	48.96%	51.35%

Thank You!